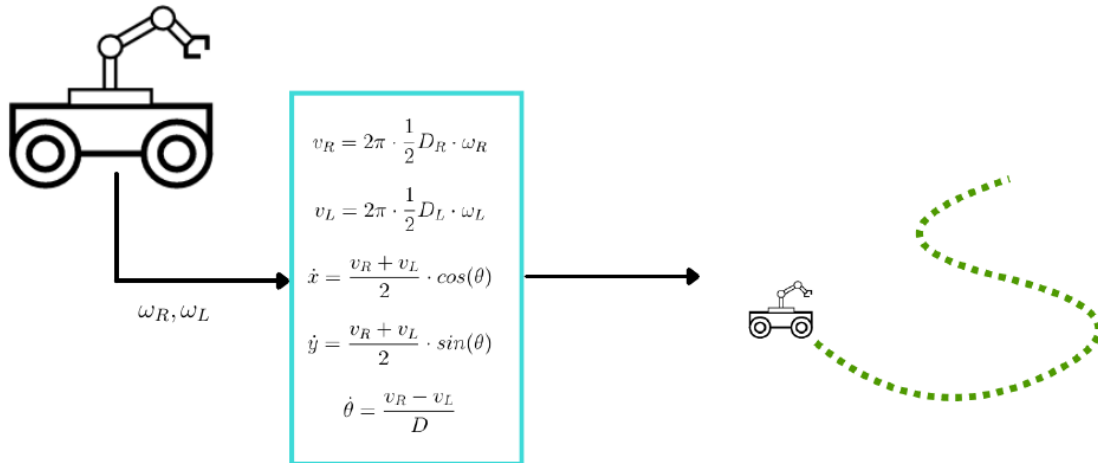


Lokalizacja lokalna (odometria)

Odometria w dziedzinie lokalizacji jest pojęciem oznaczającym przemieszczenie pojazdu wzdłuż określonej trajektorii wyznaczone na podstawie metod przyrostowych i inercyjnych w przeciwieństwie do metod absolutnych (takich jak pomiar GPS, active beacons, latarnie lokalizacyjne itp.). Odometria jest najprostszą realizacją metody zliczeniowej (ang. Dead reckoning) polegającej na wyznaczaniu (estymowaniu) pozycji pojazdu w oparciu o dane dotyczące jego prędkości i kierunku ruchu.



Po włączeniu robota i wywołaniu komendy:

```
roslaunch turtlebot3_bringup turtlebot3_robot.launch (uruchomiony na robocie)
```

w topicu `joint_states` w tablicy `velocities` będą dostępne do odczytu prędkości kół robota.

Ćwiczenie 1.

Stworzyć plik w `jupyter\_notebook` (można skorzystać z jednego z przykładów z poprzednich zajęć), w którym należy odczytać te prędkości i samodzielnie zaimplementować model kinematyki dla robota z napędem różnicowym, który prędkości przeliczy na aktualną pozycję robota. Rezultat można opublikować w topicu lub wyświetlić w terminalu.

Lokalizacja robota na zbudowanej mapie

Jedną z najbardziej popularnych a przy tym stosunkowo nowych implementacji nieparametrycznej filtracji Bayesa jest algorytm Monte Carlo Localization (MCL). Nieparametryczna ponieważ pewność (belief) $bel(x)$ w lokalizacji metodą MCL wyrażona jest nie w formie sparametryzowanej funkcji gaussowskiej lub jakiegokolwiek innej ale w formie serii M cząstek. Algorytm wywodzi się z wnioskowania bayesowskiego, ponieważ każdy stan zależy bezpośrednio tylko od stanu poprzedniego i aktualnego sterowania, czyli tak jak w przypadku lokalizacji Markowa. W przypadku Markowa cały rozkład prawdopodobieństwa był przesuwany zgodnie z przyjętym modelem ruchu. Podobnie w MCL przesuwane są cząstki. Zestaw podstawowych kroków jest bardzo podobny, zachowany jest podział na fazy: predykcji i korekcji. Cząstka w lokalizacji MCL to pojedyncza, indywidualna estymata stanu. Cząstki na początku generowane są losowo. Składają się z czterech parametrów: współrzędnych pozycji x i y , orientacji θ oraz wagi ω . Waga cząstki odzwierciedla prawdopodobieństwo jej pokrycia się z rzeczywistą pozycją robota. Wagi wszystkich cząstek sumują się do 1. Jeżeli znana jest dokładnie orientacja robota bo posiada on absolutny pomiar orientacji to może być ona wykorzystana do przypisania orientacji cząstkom i wtedy losowe generowanie cząstek ogranicza się do wygenerowania pozycji x

i y. Im więcej cząstek jest wytworzonych tym lepiej estymują one położenie robota i szybciej filtr działa (mniej iteracji jest wymaganych do znalezienia lokalizacji) jednak odbywa się to kosztem mocy obliczeniowej. W praktyce nie powinno się generować mniej niż 100 cząstek do poprawnej pracy filtru ponieważ przy mniejszej liczbie algorytm podczas kroku zwanego resample step może przypadkowo usunąć cząstki leżące stosunkowo blisko prawdziwej pozycji robota. Filtracja MCL jako bazę wykorzystuje znaną mapę obszaru, po którym jeździ robot. Najczęściej z MCL korzysta się w środowisku zamkniętym np. w budynkach. Pozycja startowa robota nie musi być znana. Dzięki wykorzystaniu mapy algorytm potrafi zlokalizować robota globalnie. W przeciwieństwie do filtracji Kalmana nie mają znaczenia nieliniowości a do tego algorytm jest prosty w implementacji.

Przy wykorzystaniu skanera laserowego jako czujnika wykrywania przeszkód proces lokalizacji wygląda tak: na początku generowane jest M próbek (cząstek) losowo rozłożonych w całej dostępnej dla robota przestrzeni, każda z nich przesuwana jest zgodnie z modelem ruchu, następnie skan z robota (to jakie przeszkody widzi) jest porównywany z mapą i cząstki, dla których ten skan jest w jakimś stopniu podobny zyskują większe wagi kosztem wag innych cząstek (wszystkie muszą się sumować do 1). Po tych fazach odbywa się tak zwany resampling - cząsteczki o największych wagach są zostawiane na mapie a te z małymi prawdopodobieństwami są usuwane. Generowana jest nowa seria cząstek lecz już nie losowo na przestrzeni całej mapy a w okolicy cząstek o największych wagach. Po pewnej liczbie takich iteracji wszystkie cząstki powinny zejść się do położenia, które najprawdopodobniej odpowiada położeniu robota na mapie.

1. Przygotowanie

Na początku należy włączyć robota postępując zgodnie z instrukcjami zawartymi na początku instrukcji 5. Uruchamiamy program na robocie poprzez połączenie ssh.

2. Mapa

Na koniec procesu mapowania otrzymujemy mapę w postaci pliku pgm oraz plik tekstowy w formacie yaml zawierający konfigurację.

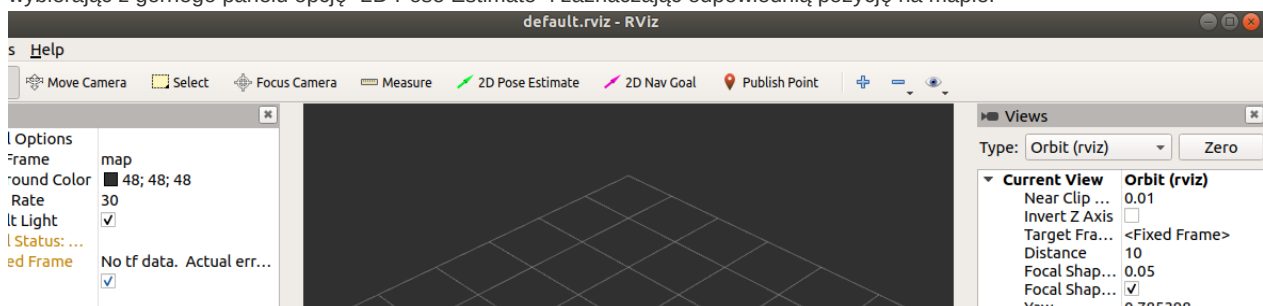
Aby wykorzystać mapę należy ją załadować do systemu używając instrukcji:

```
roslaunch map_server map_server <globalna ścieżka do pliku yaml>
```

Punkt (0,0) mapy znajduje się w miejscu, od którego rozpoczęte było mapowanie.

3. Pozycja początkowa

Z uruchomieniem robota jego pozycja lokalna (obliczona na podstawie całkowitego przemieszczenia) kół jest równa zero. Pozycja globalna (na podstawie danych ze skanera laserowego) jest również równa zero jeśli parametry algorytmu AMCL odpowiedzialne za zadanie początkowej pozycji są niezmienione. Jeżeli startujemy dokładnie z tej samej pozycji, która odpowiada punktowi (0,0) mapy oraz przy zachowaniu tej samej orientacji to nie ma problemu. Natomiast jeśli startujemy z innej pozycji to należy poinformować algorytm jaka jest pozycja początkowa (wraz z orientacją). Należy to zrobić w programie rviz wybierając z górnego panelu opcję `2D Pose Estimate` i zaznaczając odpowiednią pozycję na mapie.



3. AMCL

Uruchamiamy algorytm AMCL: (<http://wiki.ros.org/amcl>)

Zasada działania wyjaśniona tutaj: <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/12-Particle-Filters.ipynb>

W katalogu `~/catkin_ws/src/roboty_mobilne/launch` stwórz plik `navigation.launch` :

```
gedit navigation.launch
```

a następnie wklej do niego:

```
<launch>
  <!-- Arguments -->
  <arg name="model" default="$(env TURTLEBOT3_MODEL)" doc="model type [burger, waffle, waffle_pi]"/>
  <arg name="configuration_basename" default="turtlebot3_lds_2d.lua"/>
  <arg name="set_base_frame" default="base_footprint"/>
  <arg name="set_odom_frame" default="odom"/>
  <arg name="set_map_frame" default="map"/>

  <include file="$(find turtlebot3_bringup)/launch/includes/description.launch.xml">
    <arg name="model" value="$(arg model)" />
  </include>

  <node pkg="robot_state_publisher" type="robot_state_publisher" name="robot_state_publisher">
    <param name="publish_frequency" type="double" value="50.0" />
  </node>

  <!-- Arguments -->
  <arg name="scan_topic" default="scan"/>
  <arg name="initial_pose_x" default="0.0"/>
  <arg name="initial_pose_y" default="0.0"/>
  <arg name="initial_pose_a" default="0.0"/>

  <!-- AMCL -->
  <node pkg="amcl" type="amcl" name="amcl">

    <param name="min_particles" value="500"/>
    <param name="max_particles" value="3000"/>
    <param name="kld_err" value="0.02"/>
    <param name="update_min_d" value="0.20"/>
    <param name="update_min_a" value="0.20"/>
    <param name="resample_interval" value="1"/>
    <param name="transform_tolerance" value="0.2"/>
    <param name="recovery_alpha_slow" value="0.00"/>
    <param name="recovery_alpha_fast" value="0.00"/>
    <param name="initial_pose_x" value="$(arg initial_pose_x)"/>
    <param name="initial_pose_y" value="$(arg initial_pose_y)"/>
    <param name="initial_pose_a" value="$(arg initial_pose_a)"/>
    <param name="gui_publish_rate" value="50.0"/>

    <remap from="scan" to="$(arg scan_topic)"/>
    <param name="laser_max_range" value="3.5"/>
    <param name="laser_max_beams" value="180"/>
    <param name="laser_z_hit" value="0.5"/>
    <param name="laser_z_short" value="0.05"/>
    <param name="laser_z_max" value="0.05"/>
    <param name="laser_z_rand" value="0.5"/>
    <param name="laser_sigma_hit" value="0.2"/>
    <param name="laser_lambda_short" value="0.1"/>
    <param name="laser_likelihood_max_dist" value="2.0"/>
    <param name="laser_model_type" value="likelihood_field"/>

    <param name="odom_model_type" value="diff-corrected"/>
    <param name="odom_alpha1" value="0.1"/>
    <param name="odom_alpha2" value="0.1"/>
    <param name="odom_alpha3" value="0.1"/>
    <param name="odom_alpha4" value="0.1"/>
    <param name="odom_frame_id" value="odom"/>

  </node>
</launch>
```

```

<param name="base_frame_id"      value="base_footprint"/>
<param name="global_frame_id"    value="map"/>
<param name="use_map_topic"      value="true"/>

</node>

</launch>

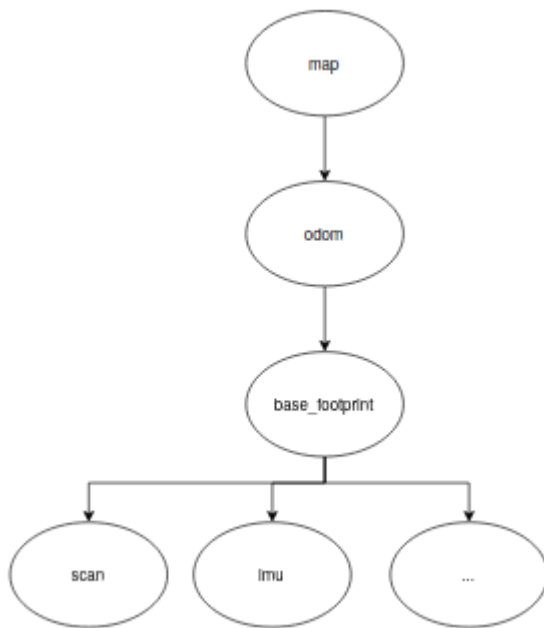
```

i zapisz.

Następnie uruchom ten plik:

```
roslaunch roboty_mobilne navigation.launch
```

Struktura przekształceń (transformat) pomiędzy układami współrzędnych powinna wyglądać tak:



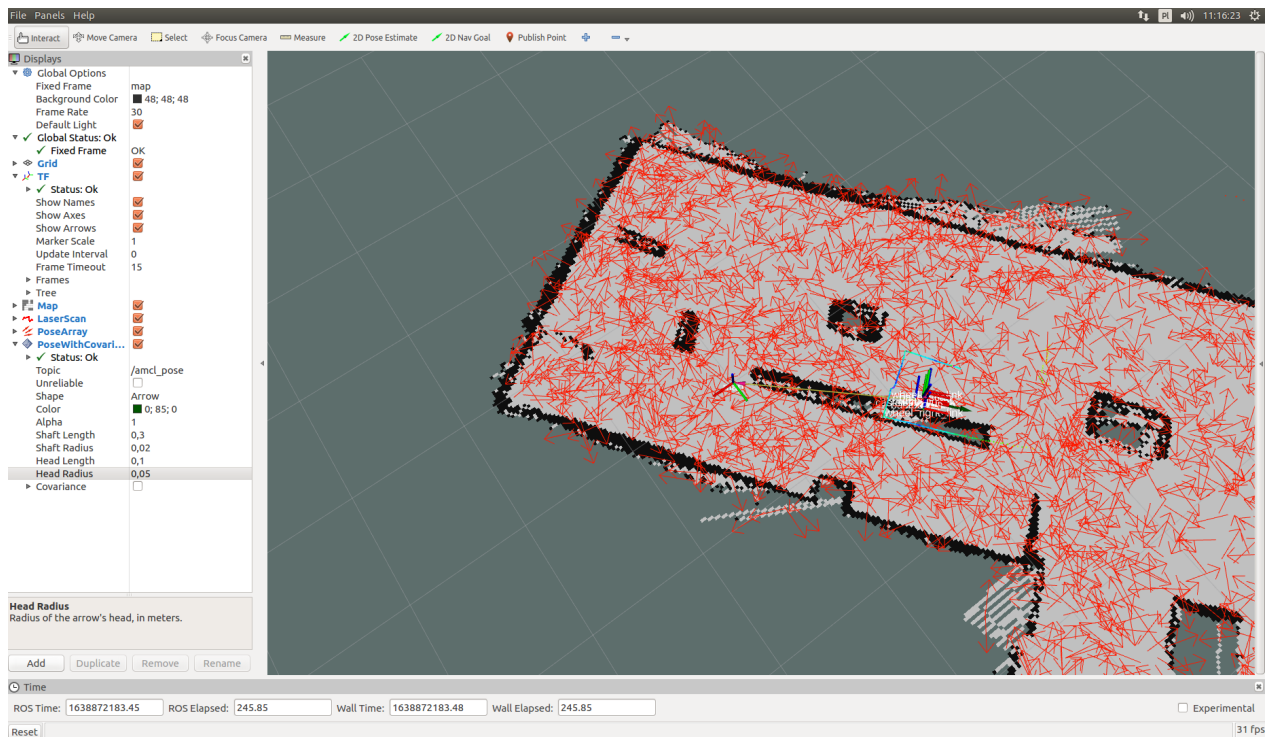
Wyświetlić ją można za pomocą komendy:

```
roslaunch roboty_mobilne navigation.launch
```

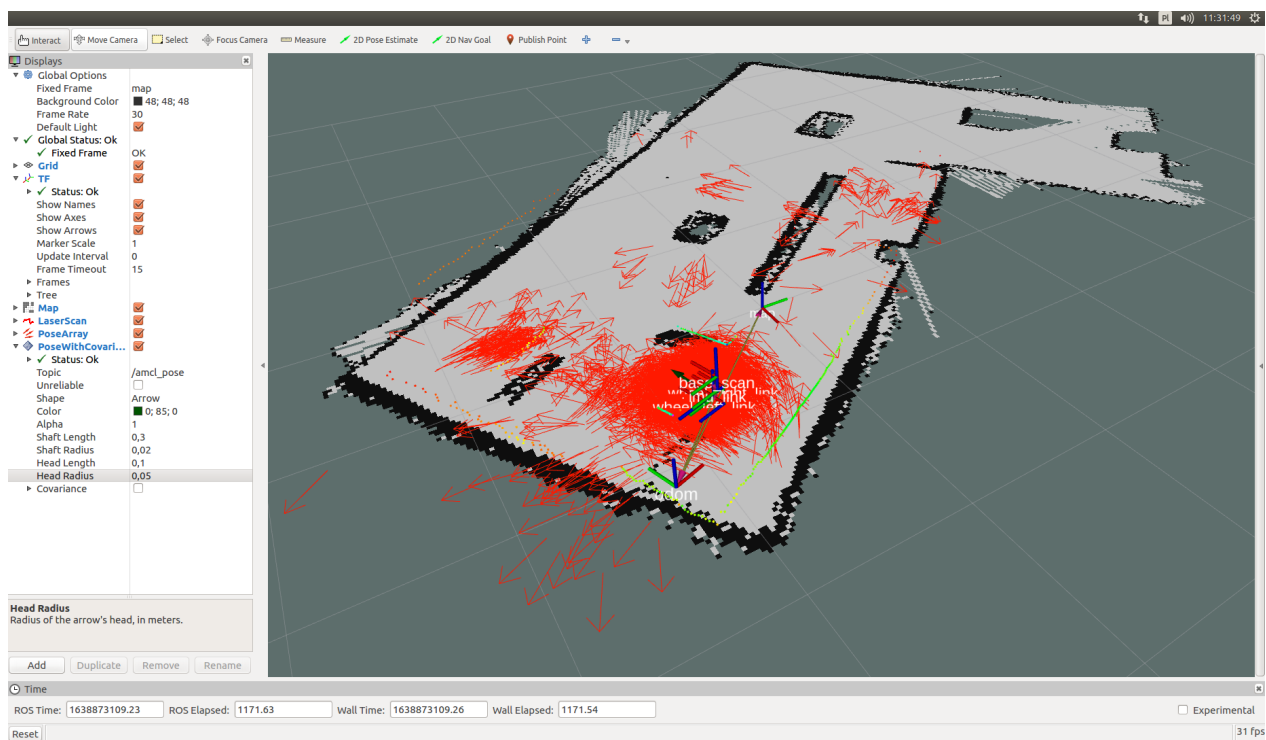
Początkowa pozycja robota jest nieznaną.

Aby wymusić wygenerowanie losowych pozycji na mapie (w liczbie określonej w pliku konfiguracyjnym jako max_points) wykorzystujemy komendę:

```
rosservice call /global_localization "{}"
```



Następnie poruszając się robotem algorytm iteracyjnie zmniejsza niepewność lokalizacji.



Ćwiczenie 2.

Przeprowadź proces lokalizacji z wykorzystaniem algorytmu AMCL aż do uzyskania konwergencji (zbieżności) do jednej grupy punktów.